

SecCall

Sistema de Gestão de Incidentes de Segurança

Relatório Técnico Final

Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas

PUC Minas — Engenharia de Software

1º Semestre 2026

Aluno: Miguel Martins Martins Fonseca da Cruz

1. Introdução

Este relatório descreve a arquitetura, as decisões de design e os resultados obtidos no desenvolvimento do SecCall, um sistema distribuído de gestão de incidentes de segurança da informação desenvolvido como Projeto Integrador da disciplina de Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas.

O sistema adota uma arquitetura orientada a eventos com dois aplicativos Flutter, um backend REST em Flask e um middleware orientado a mensagens (MOM) implementado com RabbitMQ. O nome SecCall tem como base a área escolhida, sendo uma plataforma de chamados voltada a incidentes de segurança.

O domínio escolhido fala sobre dois perfis de usuário claramente distintos:

- Cliente (Reportante): usuário que identifica e reporta um incidente.
- Prestador (Analista): profissional que recebe os chamados, investiga, atualiza o status e encerra o incidente.

2. Arquitetura do Sistema

O SecCall é composto por quatro componentes principais que se comunicam de forma síncrona e assíncrona:

O sistema é composto por diferentes componentes que trabalham de forma integrada para oferecer funcionalidades de autenticação, gerenciamento de incidentes e comunicação assíncrona entre os módulos.

O Backend Flask consiste em uma API REST desenvolvida em Python, responsável por disponibilizar os endpoints de autenticação de usuários e gerenciamento de incidentes. A API é executada na porta 5000, atuando como núcleo da aplicação e intermediando a comunicação entre os aplicativos clientes e o banco de dados.

O SQLite é utilizado como banco de dados relacional da aplicação. Os dados são armazenados localmente no arquivo incidents.db, que contém as tabelas users e incidents, responsáveis por persistir as informações dos usuários cadastrados e dos incidentes registrados.

O RabbitMQ atua como o middleware de mensageria da solução. Executado por meio de um contêiner Docker, ele gerencia a comunicação assíncrona através das filas incident.created e incident.status_updated, permitindo o desacoplamento entre os componentes do sistema.

O client_app, desenvolvido em Flutter, é a aplicação web destinada aos clientes e executada na porta 3000. Os usuários podem realizar cadastro, autenticação, abertura de novos incidentes e acompanhar o andamento de suas solicitações por meio dela

O analyst_app, também desenvolvido em Flutter e executado na porta 3001, é voltado para os analistas responsáveis pelo atendimento dos incidentes. A aplicação permite visualizar chamados, aceitá-los, recusá-los e atualizá-los até sua resolução.

Por fim, o incident_consumer.py é um processo Python executado de forma independente, responsável por consumir as mensagens publicadas nas filas do RabbitMQ. Seu objetivo é

registrar em log os eventos relacionados à criação e atualização do status dos incidentes, demonstrando o funcionamento da comunicação assíncrona implementada no sistema.

2.2 Protocolos de Comunicação

- HTTP/REST: comunicação síncrona entre os apps Flutter e o backend Flask.
- AMQP: protocolo usado pelo RabbitMQ para troca de mensagens assíncrona entre o backend e o consumidor.
- Polling HTTP (8s): o MomService nos apps Flutter faz requisições periódicas ao backend e publica resultados em StreamController.

2.3 Clean Architecture no Backend

O backend adota a separação em quatro camadas inspirada na Clean Architecture (MARTIN, 2019):

- Controller: recebe a requisição HTTP e devolve a resposta. Não contém lógica de negócio.
- Service: valida regras de negócio, orquestra o fluxo e publica eventos no MOM.
- Repository: executa comandos SQL. Único ponto de acesso ao banco de dados.
- Model: representação dos objetos de domínio (Incident, User) com serialização para JSON.

2.4 Clean Architecture no Flutter

Os apps Flutter também seguem a separação em camadas:

- Models: classes Dart com método fromJson() para desserialização das respostas da API.
- Services: IncidentService (chamadas HTTP), AuthService (autenticação) e MomService (polling assíncrono com StreamController).
- Screens: telas StatefulWidget que consomem os services e atualizam a UI via setState().

3. Arquitetura Orientada a Eventos e MOM

3.1 Eventos Implementados

incident.created	IncidentService (Flask)	incident_consumer.py	POST /api/incidents
incident.status_updated	IncidentService (Flask)	incident_consumer.py	PATCH /status

3.2 Fluxo Completo Ponta a Ponta

O fluxo completo do sistema funciona da seguinte forma:

1. Cliente cria um incidente no client_app via POST /api/incidents.
2. Backend salva no SQLite e publica o evento incident.created na fila do RabbitMQ.
3. O incident_consumer.py processa o evento de forma assíncrona e registra em log.
4. O MomService do analyst_app detecta o novo incidente via polling e exibe um banner de notificação.

- 5. Analista aceita o incidente via PATCH /api/incidents/:id/status.
- 6. Backend publica incident.status_updated no RabbitMQ.
- 7. O MomService do client_app detecta a mudança e atualiza a lista automaticamente.

4. Decisões de Design

4.1 Escolha do RabbitMQ

O RabbitMQ foi escolhido como MOM por implementar o protocolo AMQP, padrão amplamente adotado em ambientes corporativos, por possuir painel de administração visual e por ser facilmente executado via Docker com um único comando.

4.2 Polling no Flutter em vez de WebSocket

A comunicação assíncrona nos apps Flutter foi implementada via polling com intervalo de 8 segundos em vez de WebSocket direto ao RabbitMQ. Escolhi pela complexidade de manter uma conexão WebSocket estável em ambiente web e pela efetividade do polling para os requisitos de tempo real da aplicação para o escopo.

4.3 StreamController no MomService

Em vez de usar Timer.periodic diretamente nas telas, o polling foi encapsulado em um MomService com StreamController.broadcast(). Isso desacopla o mecanismo de atualização da interface permitindo que múltiplas telas assinem o mesmo stream de eventos sem criar timers redundantes.

4.4 Autenticação com Hash SHA-256

As senhas são armazenadas como hash SHA-256 no banco de dados. Não é armazenada a senha em texto puro em nenhum momento.

4.5 SQLite como Banco de Dados

O SQLite foi escolhido por não requerer instalação de servidor separado. O banco é criado automaticamente na primeira execução do backend via a função init_db(). A Clean Architecture facilita a substituição por PostgreSQL no futuro sem alterar as camadas superiores.

5. Dificuldades Encontradas e Soluções

Durante o desenvolvimento do sistema, algumas dificuldades técnicas foram identificadas e solucionadas para garantir o correto funcionamento da aplicação.

O consumo excessivo de memória pelo emulador Android em máquinas com 8 GB de RAM foi resolvido por meio da migração para o Flutter Web, utilizando o comando flutter run -d edge --web-port, sendo necessário apenas ajustar o endereço IP de 10.0.2.2 para localhost.

Problemas com caminhos contendo espaços no Windows foram solucionados utilizando aspas duplas em todos os comandos do PowerShell, como cd e Move-Item.

O erro ModuleNotFoundError durante a execução do consumidor foi corrigido com a inclusão da pasta raiz do backend no PATH do Python por meio do ajuste do sys.path utilizando os.path.abspath.

O conflito entre os dois aplicativos Flutter executando simultaneamente foi resolvido configurando portas distintas: 3000 para o `client_app` e 3001 para o `analyst_app`.

Por fim, erros nos testes automatizados causados pela renomeação de classes foram solucionados atualizando o arquivo `widget_test.dart` para referenciar corretamente as classes `SecCallApp` e `AnalystApp`.

6. Reflexão sobre os Padrões Estudados

6.1 Arquitetura Orientada a Eventos (EDA)

A utilização da arquitetura orientada a eventos permitiu desacoplar o backend dos componentes que consomem as informações. Em vez de comunicar diretamente cada sistema interessado, o backend apenas publica eventos no RabbitMQ, tornando a solução mais flexível, escalável e preparada para a inclusão de novos consumidores sem alterações na lógica principal.

6.2 Middleware Orientado a Mensagens (MOM)

O RabbitMQ foi responsável pela comunicação assíncrona entre os componentes da aplicação. A utilização de filas garantiu que os eventos fossem processados de forma organizada e confiável, reduzindo o acoplamento entre os serviços e evitando a perda de mensagens durante o processamento.

6.3 Clean Architecture

A organização do projeto em camadas facilitou a manutenção e a evolução do sistema. A separação das responsabilidades tornou o código mais modular, permitindo adicionar novas funcionalidades, como autenticação e gerenciamento de incidentes, sem impactar significativamente as demais partes da aplicação.

6.4 Arquitetura REST

A API foi desenvolvida seguindo os princípios da arquitetura REST, utilizando os métodos HTTP adequados para cada operação. Foram empregados GET para consultas, POST para criação de registros, PATCH para atualizações parciais e DELETE para exclusão, tornando a comunicação entre frontend e backend mais padronizada, clara e consistente.

7. Referências Bibliográficas

COULOURIS, George et al. Distributed Systems: concepts and design. 5. ed. Boston: Addison-Wesley, 2011.

HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

RICHARDSON, Chris. Microservices patterns: with examples in Java. Shelter Island: Manning, 2018.

BAILEY, Thomas. Flutter for beginners. 3. ed. Birmingham: Packt, 2023.